

2.2 Transpose



Core skills you'll master in this section:

- ▶ Matrix transpose: the operation of swapping rows and columns.
- ▶ Vector transpose: transposing a row vector gives a column vector, and transposing a column vector gives a row vector.
- ▶ Transposing matrices of different shapes: tall-narrow, flat, and square.
- ▶ NumPy transpose operation: use `.T` on a 2D array to get its transpose.
- ▶ Avoiding common transpose misconceptions: transpose is not a diagonal mirror, nor a left-right/up-down mirror – it is strictly a swap of rows and columns.
- ▶ Master the various properties of the transpose operation.

Matrix transpose is an especially simple yet especially important linear algebra operation. Readers can easily make mistakes with it, so we devote an entire section to explaining matrix transpose. Simply put, matrix transpose is a basic operation that swaps a matrix's rows and columns. This section first introduces the definition of transpose, illustrating the basic concept through the transpose of column vectors and row vectors. Next, we explore how matrices of different shapes change after transposition – for example, a row vector becomes a column vector, a tall-narrow matrix becomes a flat matrix, a square matrix keeps its shape after transposition, and a symmetric matrix remains unchanged under transposition. Finally, we introduce the important properties of the transpose operation, including that double transposition restores the original matrix, transpose commutes with addition, and scalar multiplication is unaffected by transpose – laying the foundation for further study of matrix operations.

What Is Transpose?

As mentioned earlier in this book, transposing a column vector gives a row vector, and vice versa.

When transpose is applied to a matrix, the effect is a new matrix obtained by swapping the matrix's rows and columns. Transpose is a “matrix $\rightarrow$ matrix” operation.

The transpose of matrix $\mathbf{A}$ is denoted $\mathbf{A}^T$ or $\mathbf{A}'$. NumPy uses `.T` to denote the array transpose operation, so this book uses only the $\mathbf{A}^T$ notation.

As shown in Figure 1, an $m \times p$ matrix $\mathbf{A}$ transposed gives a $p \times m$ matrix $\mathbf{A}^T$.

A matrix's main-diagonal elements (outlined in black) stay in the same position before and after transposition. This is because the main-diagonal elements sit where the row index equals the column index – that is, element $a_{i,i}$ remains at $a_{i,i}$.

That is, elements $a_{1,1}$, $a_{2,2}$, $a_{3,3}$, and so on, keep their positions after transposition.

If we think of each element of the matrix as a square, the whole process is equivalent to mirroring matrix A about its main diagonal (main diagonal).

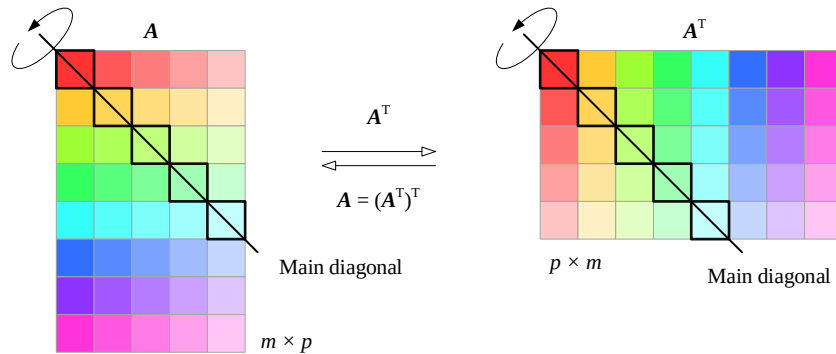


Figure 1. Matrix Transpose

In fact, matrix transpose is not complicated at all. If you think of a matrix as a table, transposing it simply swaps the table's rows and columns.

Columns $\rightarrow$ Rows

As shown in the figure below, matrix A can be written as a set of column vectors:

$$A = \begin{bmatrix} \mathbf{a}_1 & \mathbf{a}_2 & \cdots & \mathbf{a}_p \end{bmatrix}$$

Matrix A transposed, A^T , can be expanded and written as:

$$A^T = \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \\ \vdots \\ \mathbf{a}_p^T \end{bmatrix}$$

As shown in Figure 2, this means that when we express matrix A as a set of column vectors, transposing the matrix can be understood as converting each original column vector into its corresponding row vector.

For example, matrix A 's column 1 becomes row 1 of the new matrix A^T after transposition, column 2 becomes row 2, and so on.

This way, we can intuitively see how the transpose operation turns elements that were originally arranged vertically into a horizontal arrangement. The elements' relative order (the ordering of the color shades) does not change.

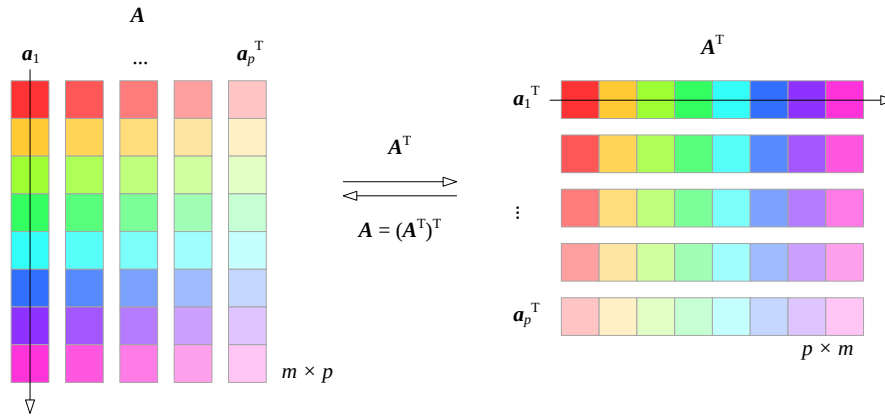


Figure 2. Matrix Transpose Turns Columns into Rows

For example,

$$A^T = \begin{bmatrix} 0 \\ 3 \\ 5 \end{bmatrix} \begin{bmatrix} 5 \\ 4 \\ 0 \end{bmatrix}^T = \begin{bmatrix} 0 & 3 & 5 \\ 5 & 4 & 0 \end{bmatrix}$$

In the original matrix A , the 1st element of each column becomes the 1st element of each row in A^T after transposition.

The 2nd element of each column in the original matrix becomes the 2nd element of each row in the transposed matrix.

Next, let's use NumPy to perform the transpose operation on a matrix (2D array); the code is shown in Code 1. Let's walk through its key lines.

You should already be familiar with [a](#) – these two lines. As mentioned in the previous section, `numpy.random.seed()` sets the seed for random number generation, ensuring the results are reproducible – that is, the same random array is produced every time the code runs. Next, `numpy.random.randint(0, 10, (5, 12))` generates a 2D array of size 5×12 . `randint(0, 10)` means generating random integers between 0 and 9.

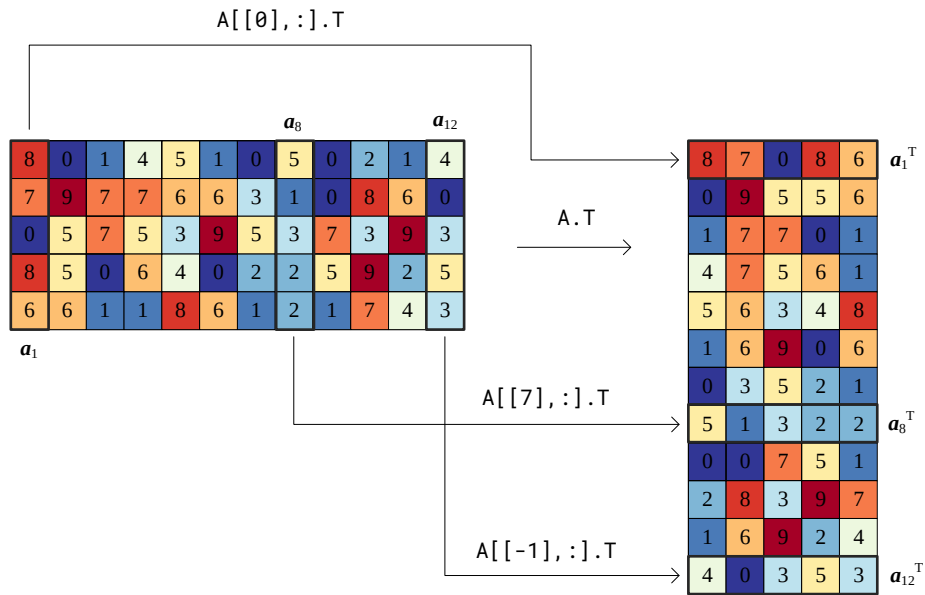
[b](#) Compute the transpose of the 2D array (matrix A). `".T"` is the NumPy method for getting a matrix's transpose.

[c](#) Use `seaborn.heatmap()` to create a heatmap.

`A[[0], :]` selects matrix A 's first row, and the result is a 2D array, as shown in Figure 3. Figure 3 also shows the transposes of several other rows.

`cmap = 'RdYIBu_r'` specifies the heatmap's colormap;
`square = True` keeps each cell of the heatmap square;
`cbar = False` disables the color bar on the right;
`annot = True` displays the numeric value in each cell;
`vmin = 0` and `vmax = 9` set the value range shown in the heatmap;
`linecolor = 'w'` sets the border color between cells to white;
`linewidths = 0.25` sets the width of the border lines between cells.

[d](#) Similar to [c](#). The difference is that [c](#) displays the transpose of matrix A 's first row, `A[[0], :].T`, so the result shows the first row's data displayed as a column vector.

Figure 3. NumPy 2D Array Transpose, Columns $\rightarrow$ RowsCode 1. NumPy 2D Array Transpose, Columns $\rightarrow$ Rows | LA_02_02_01.ipynb

```

## 初始化
import numpy as np
import seaborn as sns

a np.random.seed(88)
A = np.random.randint(0, 10, (5, 12))

b ## 计算矩阵 A 的转置
A_T = A.T

c ## 行向量的转置
sns.heatmap(A[[0],:], cmap = 'RdYlBu_r', square = True,
            cbar = False, annot = True, vmin = 0, vmax = 9,
            linecolor = 'w', linewidths = 0.25)

d sns.heatmap(A[[0],:].T, cmap = 'RdYlBu_r', square = True,
            cbar = False, annot = True, vmin = 0, vmax = 9,
            linecolor = 'w', linewidths = 0.25)

```

Rows $\rightarrow$ Columns

Conversely, matrix A can be written as a set of row vectors:

$$A = \begin{bmatrix} \mathbf{a}^{(1)} \\ \mathbf{a}^{(2)} \\ \vdots \\ \mathbf{a}^{(m)} \end{bmatrix}$$

$\mathbf{A}^T$ can then be written as:

$$\mathbf{A}^T = \begin{bmatrix} \mathbf{a}^{(1)T} & \mathbf{a}^{(2)T} & \dots & \mathbf{a}^{(m)T} \end{bmatrix}$$

As shown in Figure 4, once the matrix is written as a set of row vectors, transposing converts each row vector into its corresponding column vector. However, the elements' relative positions (the light-to-dark gradient within the same color) do not change.

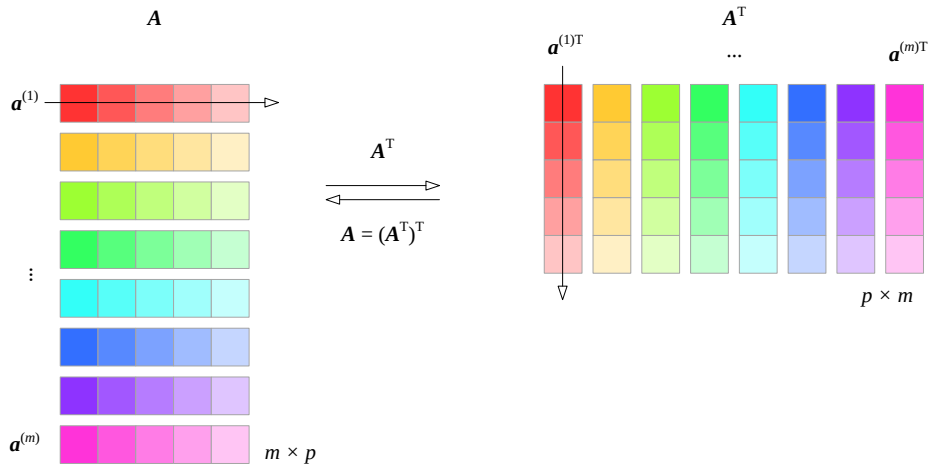


Figure 4. Matrix Transpose Turns Rows into Columns

Returning to the earlier example,

$$\mathbf{A}^T = \begin{bmatrix} \begin{bmatrix} 0 & 5 \end{bmatrix} \\ \begin{bmatrix} 3 & 4 \end{bmatrix} \\ \begin{bmatrix} 5 & 0 \end{bmatrix} \end{bmatrix}^T = \begin{bmatrix} \begin{bmatrix} 0 \\ 3 \\ 5 \end{bmatrix} & \begin{bmatrix} 5 \\ 4 \\ 0 \end{bmatrix} \end{bmatrix}$$

In the original matrix $\mathbf{A}$, the 1st element of each row becomes, after transposition, the 1st element of each column in $\mathbf{A}^T$.

The 2nd element of each row becomes, after transposition, the 2nd element of each column in $\mathbf{A}^T$.

By extension, all the elements in row i of the original matrix become the corresponding elements in column i of the transposed matrix. The essence of transpose is swapping the matrix's rows and columns — converting an arrangement of row vectors into an arrangement of column vectors.

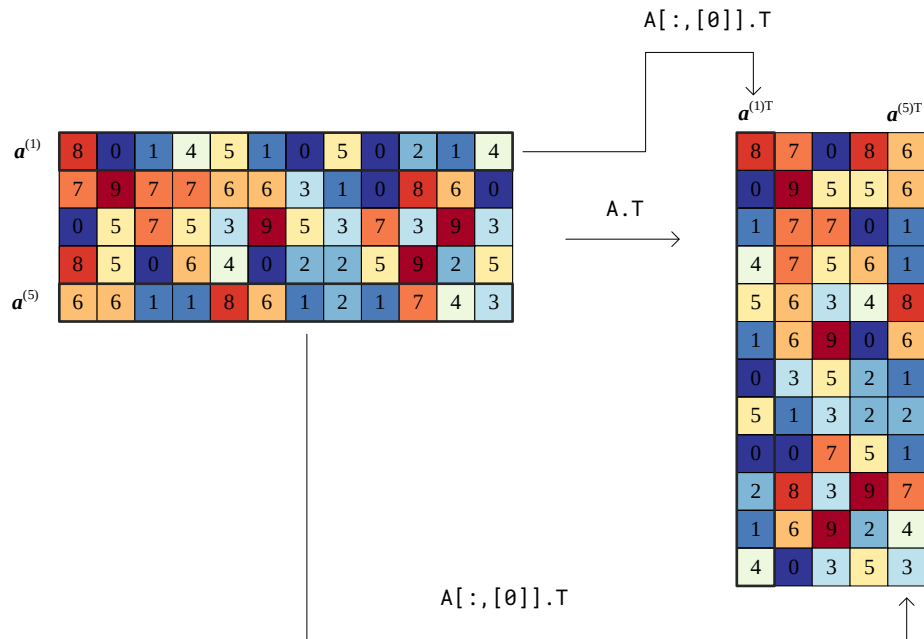


Figure 5. NumPy 2D Array Transpose, Rows → Columns



LA_02_02_01.ipynb also visualizes how matrix A 's row vectors are transposed into column vectors, specifically as shown in Figure 5.

Transposing Matrices of Different Shapes

Next, let's summarize how the shape changes when transposing matrices of different shapes.

As shown in Figure 6, the transpose of a row vector is a column vector, for example

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}^T = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

the transpose of a column vector is a row vector, for example

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}^T = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

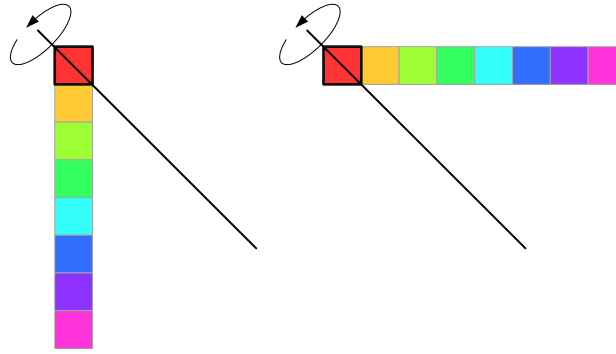


Figure 6. Transposing Row and Column Vectors into Each Other

As shown in Figure 1, transposing a tall-narrow matrix (more rows than columns) gives a flat matrix (fewer rows than columns):

$$\begin{bmatrix} 1 & 5 \\ 2 & 6 \\ 3 & 7 \\ 4 & 8 \end{bmatrix}^T = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}$$

Conversely, transposing a flat matrix gives a tall-narrow matrix:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}^T = \begin{bmatrix} 1 & 5 \\ 2 & 6 \\ 3 & 7 \\ 4 & 8 \end{bmatrix}$$

As shown in Figure 7, the transpose of a square matrix is still a square matrix:

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}^T = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

but the elements' positions are mirrored about the main diagonal.

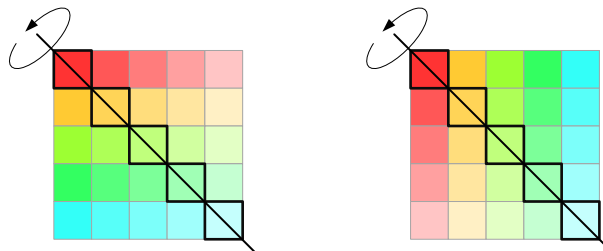


Figure 7. Transpose of a Square Matrix

As shown in Figure 8, the transpose of a symmetric matrix equals itself, for example

$$\begin{bmatrix} 1 & 8 \\ 8 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 8 \\ 8 & 6 \end{bmatrix}$$

That is, if matrix A is symmetric, then it satisfies $A = A^T$. Matrix shape is the topic of the next section.

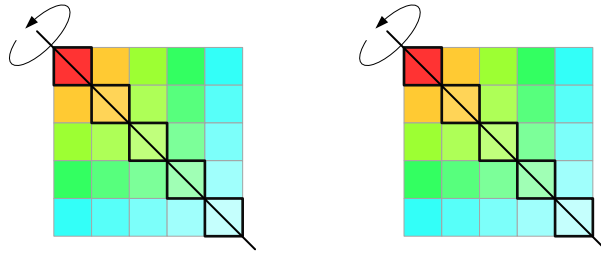


Figure 8. Transpose of a Symmetric Matrix

After transposing matrices of different shapes, the shape may change, but the core rule is always the same: rows become columns and columns become rows; moreover, the elements' relative positions do not change.



LA_02_02_02.ipynb computes the transpose of matrices of different shapes – please study it on your own.

A Geometric Perspective

As shown in Figure 9, A 's row vectors can be viewed as three arrows in a plane, while A 's column vectors can be viewed as two arrows in 3D space.

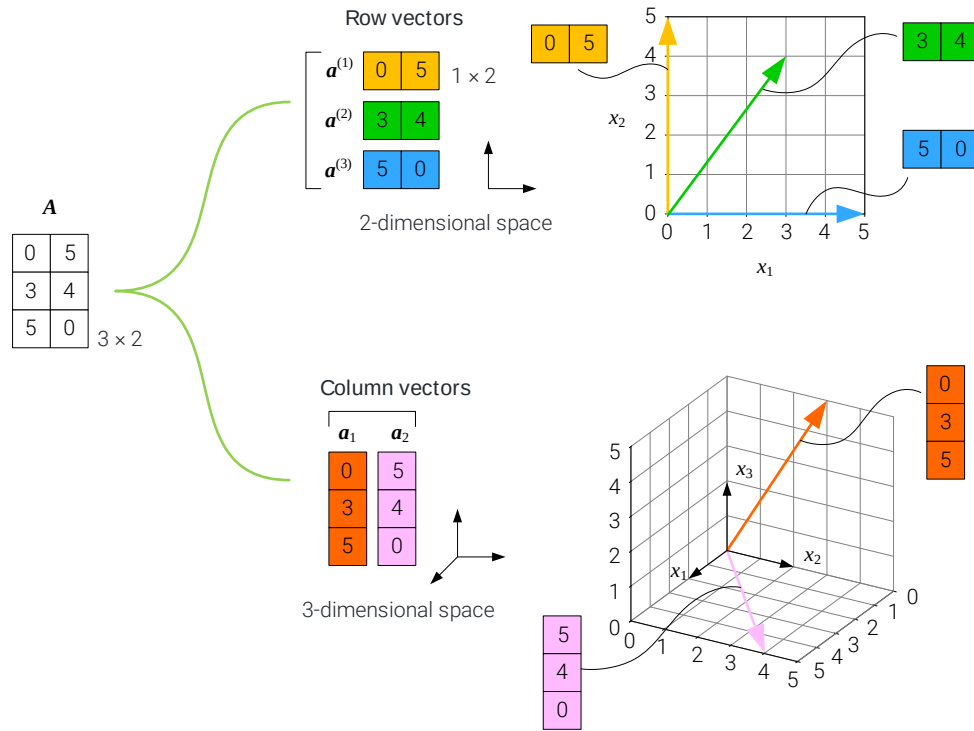
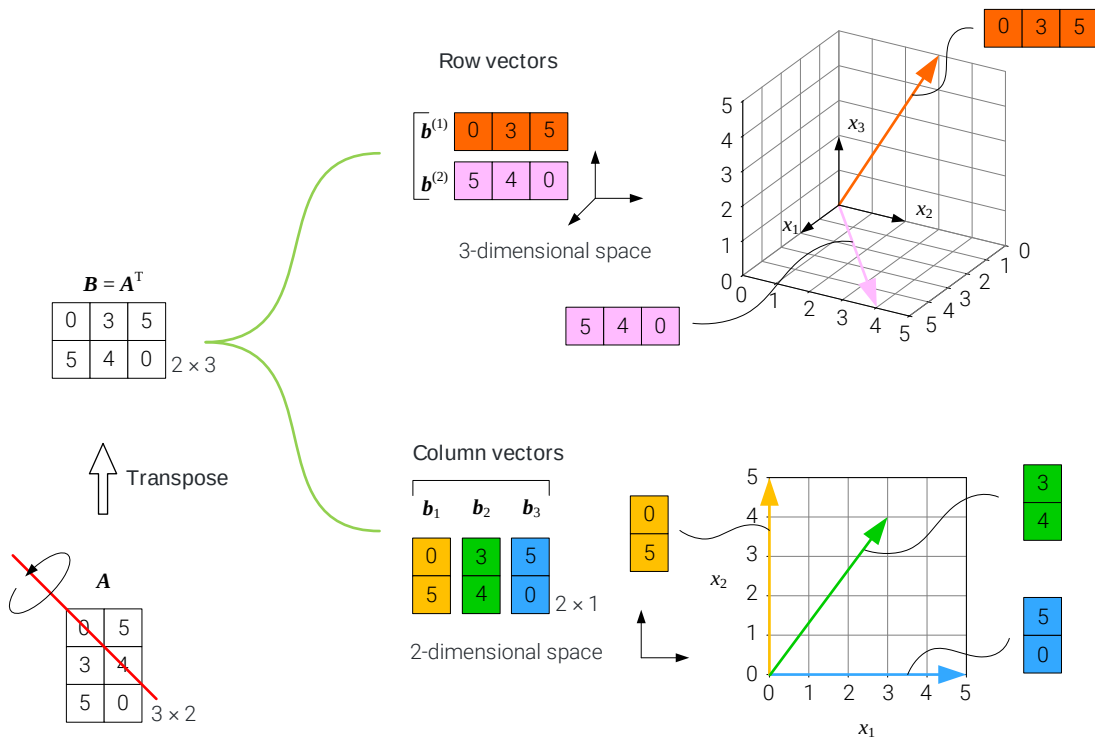
Figure 9. Matrix A 's Row Vectors and Column Vectors, Image from The Beauty of Visualization

Figure 10 shows the row and column vectors of the transposed matrix A^T . Here, A^T 's row vectors are two arrows in 3D space, while A^T 's column vectors are three arrows in the plane.

Figure 10. Matrix A 's Row Vectors and Column Vectors After Transposition, Image from The Beauty of Visualization

Things to Watch Out For

Matrix transpose is not a simple mirroring operation — it is a precise process of swapping rows and columns. Beginners should pay close attention to the following points.

“Main diagonal” is not the same as “diagonal.” Matrix transpose is not a mirror across the diagonal. As shown in Figure 11 (a), for a non-square matrix, mirroring along the diagonal doesn't even make sense.

Only for a square matrix (rows equal to columns) does transpose look like a mirror across the diagonal, and that is just a special case. Unlike the diagonal, a matrix's main diagonal refers to the line formed by the elements whose row index equals their column index. As mentioned earlier, matrix A 's main diagonal is the line formed by the elements a_{ii} .

Matrix transpose is also not a left-right mirror, as shown in Figure 11 (b). Before and after transposition, the order of the columns is converted into the order of the rows — that is, transpose strictly maps the elements of row i to column i . For example, if the original matrix's column 1 is on the left, after transposition it becomes row 1, rather than moving to the far right.

Matrix transpose is also not an up-down mirror, as shown in Figure 11 (c). A row near the top of the matrix does not move to the bottom just because of transposition.

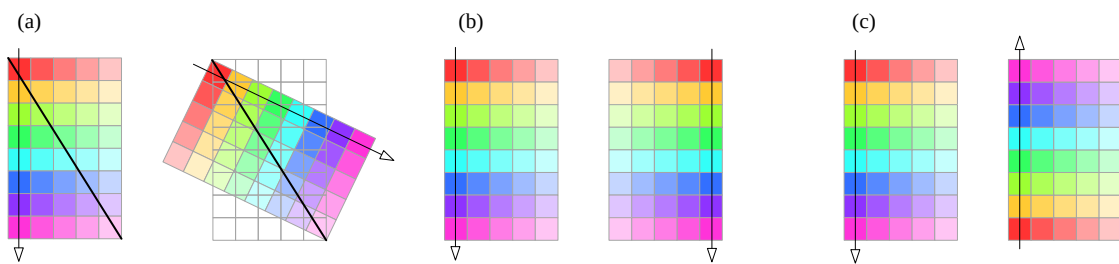


Figure 11. Common Misconceptions About Matrix Transpose

Important Properties

The following important properties of matrix transpose deserve your attention.

Double transposition restores the original matrix, that is,

$$(A^T)^T = A$$

By definition, matrix transpose turns rows into columns and columns into rows. If we transpose an already-transposed matrix again, the original row/column structure is restored. For example, given the original matrix

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \\ a_{3,1} & a_{3,2} \end{bmatrix}$$

Transposing matrix A gives

$$\mathbf{A}^T = \begin{bmatrix} a_{1,1} & a_{2,1} & a_{3,1} \\ a_{1,2} & a_{2,2} & a_{3,2} \end{bmatrix}$$

$\mathbf{A}^T$ transposing again gives

$$(\mathbf{A}^T)^T = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \\ a_{3,1} & a_{3,2} \end{bmatrix}$$

The transpose operation commutes with matrix addition, that is,

$$(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$$

Matrix addition is performed element by element, so the transpose operation does not affect the relative position of each element in the sum.

For example, given the following two matrices of the same shape

$$\mathbf{A} = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \\ a_{3,1} & a_{3,2} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \\ b_{3,1} & b_{3,2} \end{bmatrix}$$

Add first, then transpose — that is, compute $(\mathbf{A} + \mathbf{B})$

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} a_{1,1} + b_{1,1} & a_{1,2} + b_{1,2} \\ a_{2,1} + b_{2,1} & a_{2,2} + b_{2,2} \\ a_{3,1} + b_{3,1} & a_{3,2} + b_{3,2} \end{bmatrix}$$

Transposing the result, that is, $(\mathbf{A} + \mathbf{B})^T$

$$(\mathbf{A} + \mathbf{B})^T = \begin{bmatrix} a_{1,1} + b_{1,1} & a_{2,1} + b_{2,1} & a_{3,1} + b_{3,1} \\ a_{1,2} + b_{1,2} & a_{2,2} + b_{2,2} & a_{3,2} + b_{3,2} \end{bmatrix}$$

First transpose $\mathbf{A}$ and $\mathbf{B}$ separately

$$\mathbf{A}^T = \begin{bmatrix} a_{1,1} & a_{2,1} & a_{3,1} \\ a_{1,2} & a_{2,2} & a_{3,2} \end{bmatrix}, \quad \mathbf{B}^T = \begin{bmatrix} b_{1,1} & b_{2,1} & b_{3,1} \\ b_{1,2} & b_{2,2} & b_{3,2} \end{bmatrix}$$

then add $\mathbf{A}^T$ and $\mathbf{B}^T$ together

$$\mathbf{A}^T + \mathbf{B}^T = \begin{bmatrix} a_{1,1} + b_{1,1} & a_{2,1} + b_{2,1} & a_{3,1} + b_{3,1} \\ a_{1,2} + b_{1,2} & a_{2,2} + b_{2,2} & a_{3,2} + b_{3,2} \end{bmatrix}$$

The computation shows that whether we add first and then transpose, or transpose first and then add, the final result is the same.

Scalar multiplication does not affect transpose, that is,

$$(k\mathbf{A})^T = k\mathbf{A}^T$$

Here, k is a scalar. Scalar multiplication scales every element of the matrix by the same factor, while transpose only changes the positions of elements without affecting their values — so scalar multiplication and transpose can be performed in either order.

For example, multiplying by the scalar first gives

$$k\mathbf{A} = \begin{bmatrix} ka_{1,1} & ka_{1,2} \\ ka_{2,1} & ka_{2,2} \\ ka_{3,1} & ka_{3,2} \end{bmatrix}$$

Transposing $k\mathbf{A}$ gives

$$(k\mathbf{A})^T = \begin{bmatrix} ka_{1,1} & ka_{2,1} & ka_{3,1} \\ ka_{1,2} & ka_{2,2} & ka_{3,2} \end{bmatrix}$$

Taking a different computation path, first transpose $\mathbf{A}$ to get

$$\mathbf{A}^T = \begin{bmatrix} a_{1,1} & a_{2,1} & a_{3,1} \\ a_{1,2} & a_{2,2} & a_{3,2} \end{bmatrix}$$

then multiply the result by the scalar k

$$k\mathbf{A}^T = \begin{bmatrix} ka_{1,1} & ka_{2,1} & ka_{3,1} \\ ka_{1,2} & ka_{2,2} & ka_{3,2} \end{bmatrix}$$

The computation shows that whether we scalar-multiply first and then transpose, or transpose first and then scalar-multiply, the final matrix is the same.

 Note that once transpose is combined with matrix multiplication and matrix inverses, there are a few more properties worth noting – but we don't need to worry about those just yet.



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. LA_02_01_01.ipynb repeatedly calls `seaborn.heatmap()` to draw similar heatmaps. Please rewrite the repeated code as a single Python function for convenient reuse in later visualizations.

Q2. Use a random number generator to create a 5×8 matrix, then extract each row of the matrix; finally, use `numpy.block()` to reassemble these row vectors back into the original matrix.

<https://numpy.org/doc/2.2/reference/generated/numpy.block.html>

Q3. Write Python code to verify that “double transposition restores the original matrix”; use `numpy.array_equal()` to check whether the matrices are equal.

Q4. Write Python code to verify that “the transpose operation commutes with matrix addition.”

Q5. Write Python code to verify that “scalar multiplication does not affect transpose.”

Q6. Write Python code to try to reproduce Figure 9 and Figure 10.